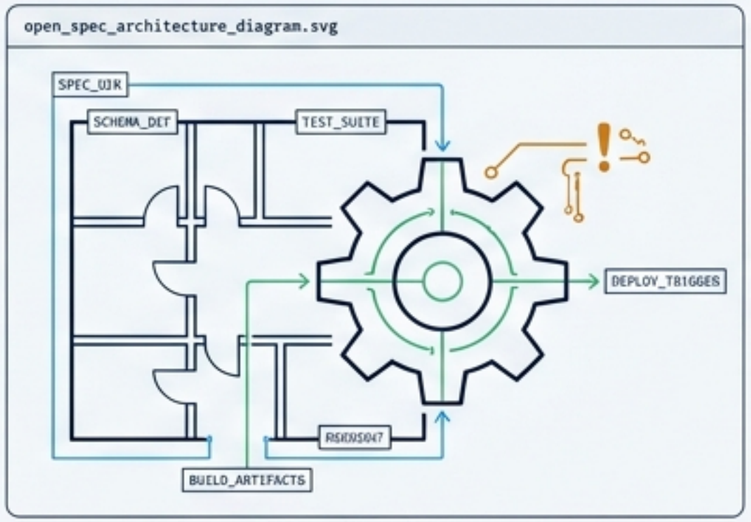
# OpenSpec: Dissecando o Spec-Driven Development

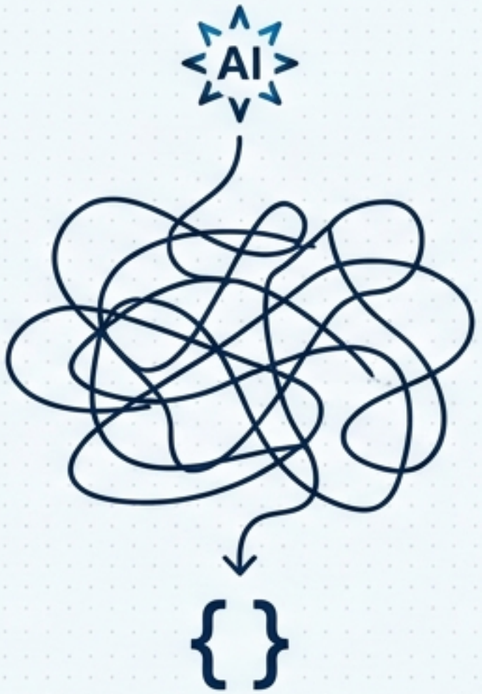
O guia estrutural sobre pastas, artefatos e fluxos de trabalho para engenharia de software previsível com Inteligência Artificial.




Gemini Notebook

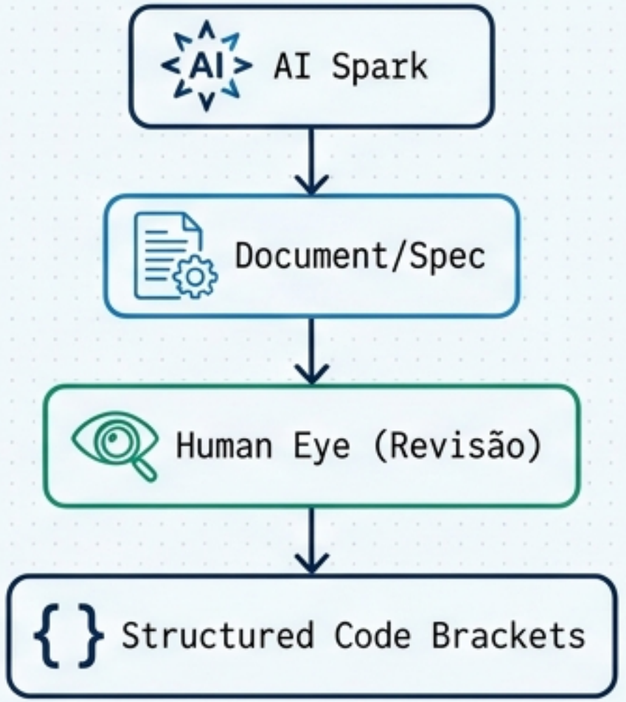
# O Paradigma: Antes vs. Agora

## O Paradigma Antigo




**IA Direto para o Código:** Falta de previsibilidade, retrabalho constante e ausência de histórico. A IA tenta adivinhar a arquitetura.

## A Abordagem SDD - Spec-Driven Development

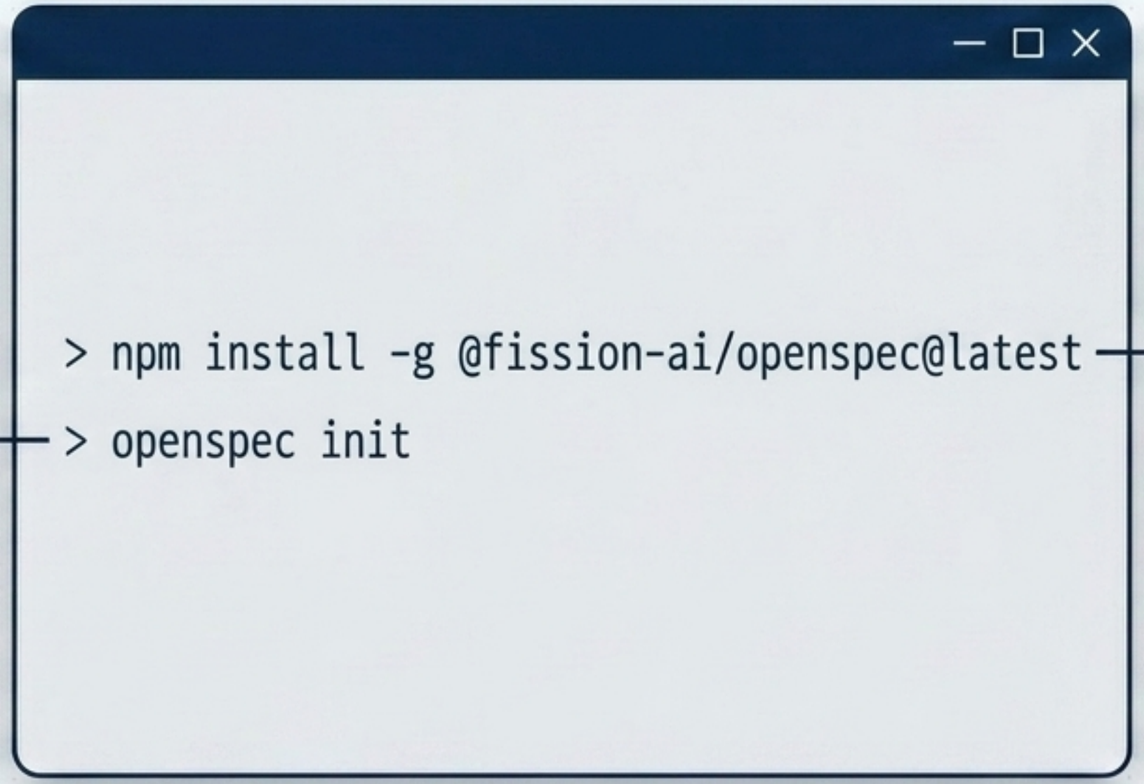



**Especificação Primeiro:** Clareza estrutural, confiabilidade e documentação viva perfeitamente sincronizada com o código final.

**A Regra de Ouro do OpenSpec:**
A especificação sempre vem antes da implementação.

Gemini Notebook

# Preparando o Terreno




Instalação global. Requer Node.js v20.19.0+. Sem necessidade de chaves de API ou ferramentas MCP.

Constrói a estrutura de pastas do Spec-Driven Development e injeta as ferramentas operacionais no ambiente do projeto.

Gemini Notebook

# A Arquitetura de Duas Camadas

## .claude/
### A Camada Operacional

- **Função:** Ensina a IA COMO trabalhar. É o motor temporário e utilitário.
- **Conteúdo:** Comandos rápidos, workflows, diretrizes de prompt e skills (ferramentas permitidas).



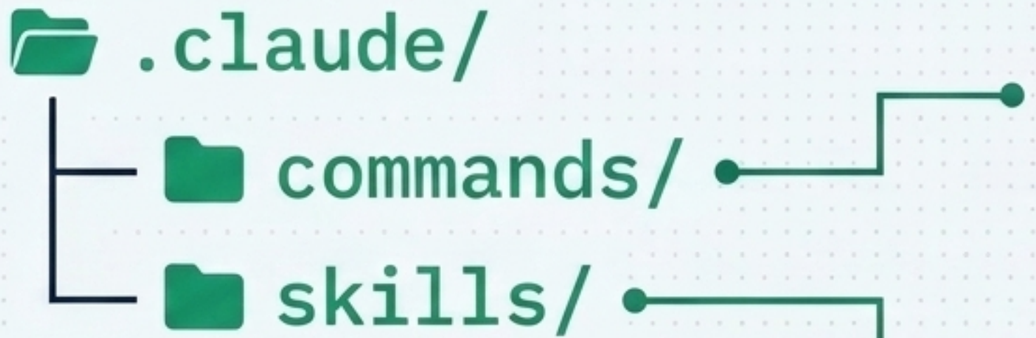
AI Agent

## .openspec/
### A Camada Arquitetural

- **Função:** Armazena o CONHECIMENTO do projeto. É o cérebro permanente.
- **Conteúdo:** Especificações consolidadas, laboratório de mudanças (changes), arquivo histórico e configurações globais.

Gemini Notebook

# Deep Dive: A Árvore Operacional



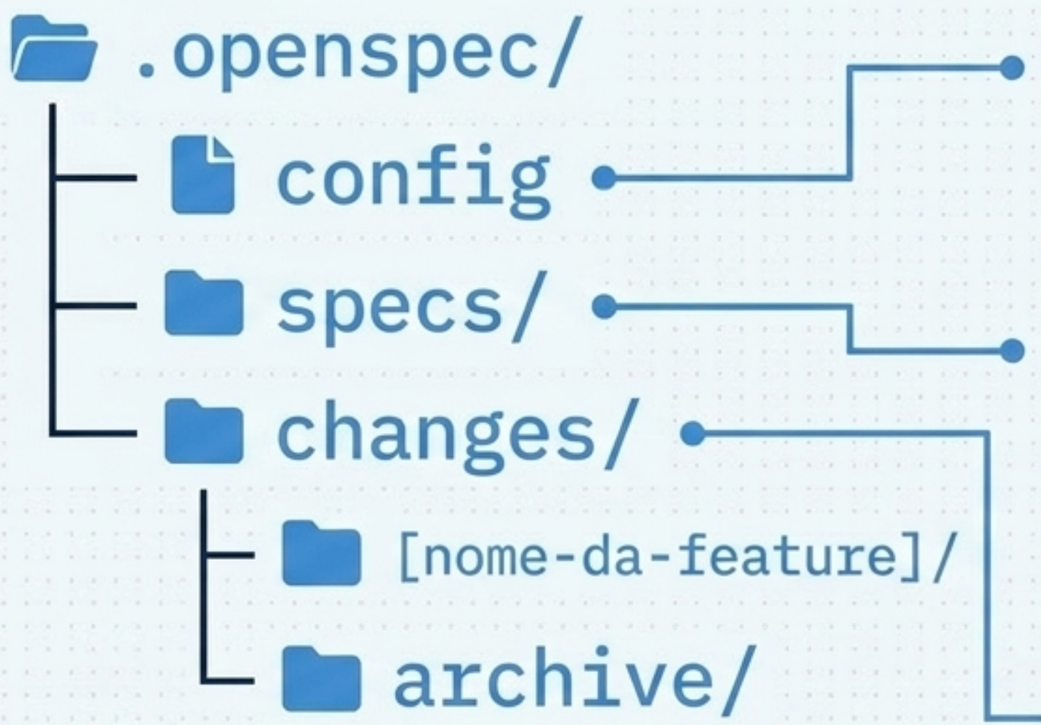

Contém os atalhos rápidos acionados no terminal da IA (ex: Claude Code) através do prefixo /opsx.

Define as ferramentas e habilidades que a IA tem permissão para usar dentro do escopo deste projeto específico.

**Conceito Chave:** Esta pasta é utilitária. Ela define os limites e capacidades da IA, garantindo que o agente opere de forma controlada e sem improvisos.

Gemini Notebook

# Deep Dive: A Árvore Arquitetural




Arquivo central com as regras globais e contexto do OpenSpec.

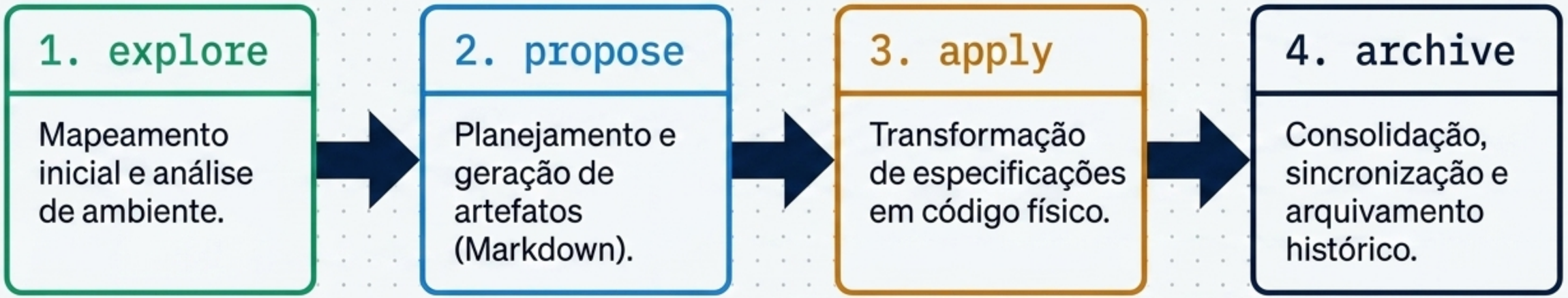
A Fonte da Verdade. Onde a documentação oficial, permanente e consolidada reside.

O **Laboratório**. Contém propostas ativas em andamento e o histórico de tudo que já foi implementado. 90% do tempo de desenvolvimento ativo ocorre aqui.

Gemini Notebook

# O Ciclo de Vida da Funcionalidade

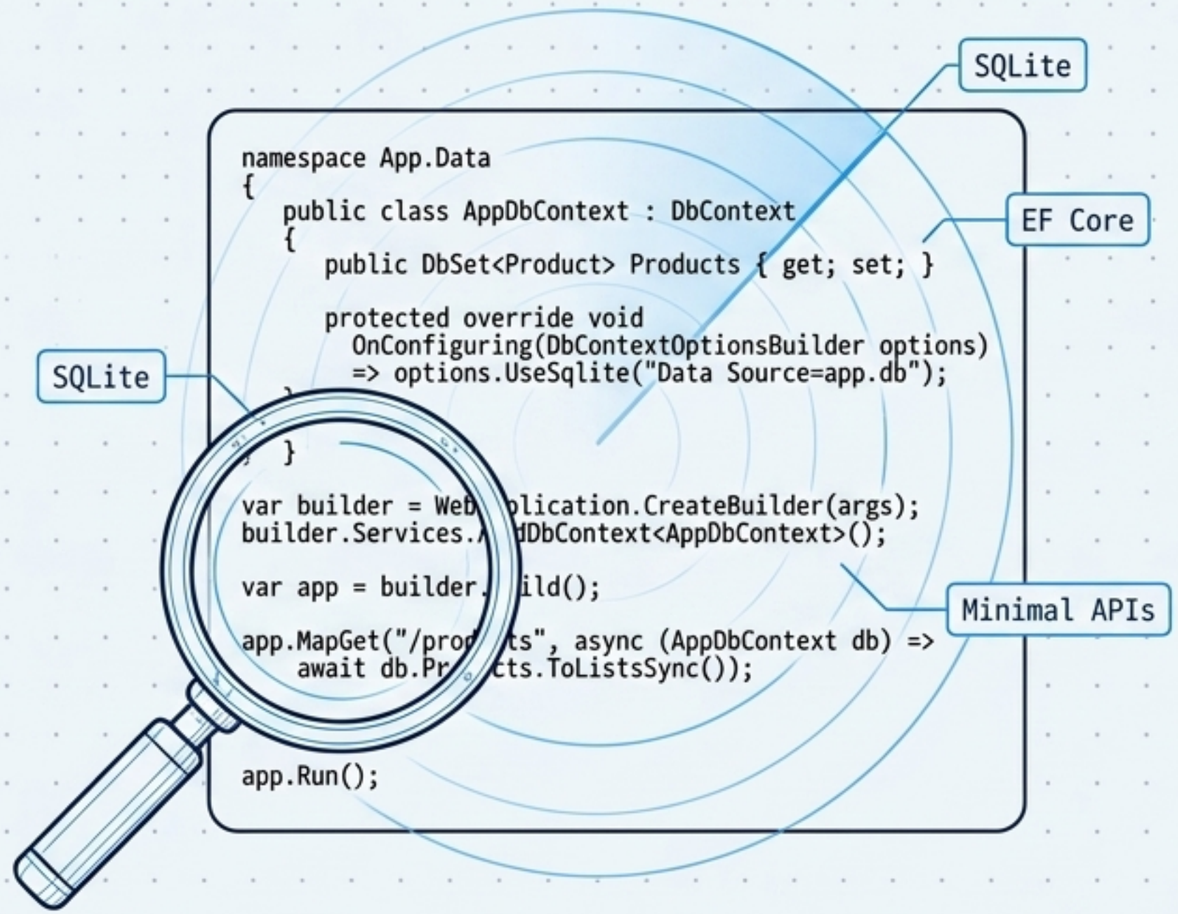
## O Fluxo de Trabalho Padrão (Prefixo /opsx)




Gemini Notebook

# Etapa 1: O Mapeamento Contextual

## O Fluxo de Trabalho Padrão (Prefixo **/opsx**)




### Comando: /opsx explorer

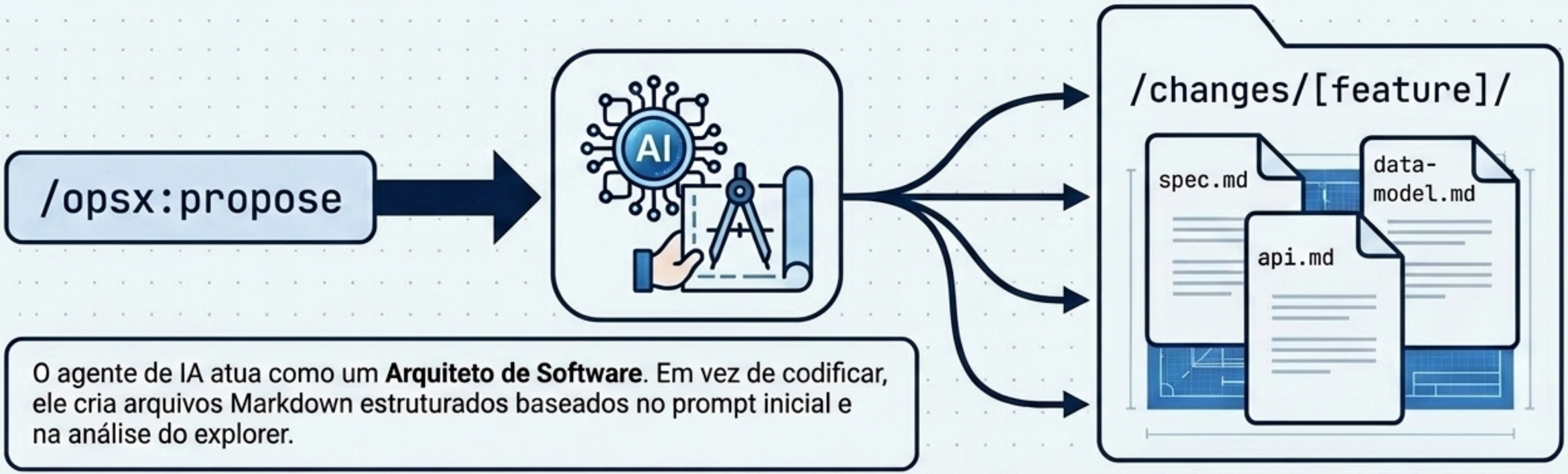
A IA analisa o ambiente atual, identifica tecnologias instaladas, mapeia o banco de dados e gera um entendimento inicial da arquitetura.

| Projetos Legados | Projetos Novos |
| --- | --- |
| Indispensável para compreender dependências e código existente antes de tocar na base. | Ajuda a investigar a viabilidade técnica e a refinar requisitos arquiteturais junto com a IA antes da especificação. |

Gemini Notebook

# Etapa 2: A Geração de Artefatos

## O Fluxo de Trabalho Padrão (Prefixo **/opsx**)




O agente de IA atua como um **Arquiteto de Software**. Em vez de codificar, ele cria arquivos Markdown estruturados baseados no prompt inicial e na análise do explorer.

**PONTO DE REVISÃO HUMANA:** É neste momento que o desenvolvedor deve ler os arquivos gerados, iterar com a IA (ex: 'Ajuste o Swagger', 'Mude o status code para 409') e congelar o escopo da funcionalidade.

Gemini Notebook

# A Anatomia do Planejamento

## proposal.md

O **‘Por Que’** e o **‘O Que’**. Define a motivação, o impacto no sistema, o escopo da API e as limitações estruturais da funcionalidade.

## design.md

As **Decisões Técnicas**. Mapeia a arquitetura escolhida, organização de pastas, contratos de endpoints e regras de negócio essenciais.

## specs.md

**Os Requisitos**. A listagem rigorosa de requisitos de sistema e múltiplos cenários testáveis detalhando comportamentos de sucesso e falha.

## tasks.md

O **Checklist Operacional**. Uma lista de tarefas técnicas minuciosas e ordenadas por dependência de execução. O motor da próxima fase.

Gemini Notebook

# Etapa 3: A Transformação Física

**tasks.md**

[x] Implementar Modelo de Dados
[x] Criar Repositório
[ ] Desenvolver Controlador API
[ ] Escrever Testes Unitários

**/opsx: apply**

**/src/**

**Controller.cs:**

```
[HttpGet]
public IActionResult Get() {
  return Ok(new { status = "active" });
}
```

**Service.cs:**

```
public class Service : IService {
  ...
}
```

**Repository.cs:**

```
public async Task<Entity> FindById(int id) {
  ...
}
```

## A Mecânica:

A IA assume o papel de Programador. Ela segue estritamente os artefatos gerados, lendo, executando e gerando o código físico.

## Atualização em Tempo Real:

Enquanto programa, a IA atualiza o tasks.md automaticamente, marcando as tarefas concluídas, mantendo o desenvolvedor ciente do status exato da implementação.

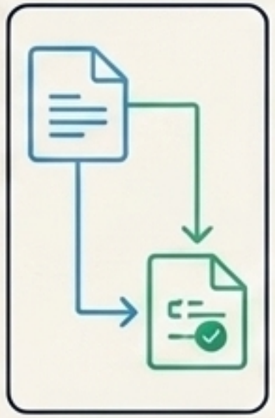
Gemini Notebook

# Etapa 4: A Encruzilhada da Sincronização

/changes/

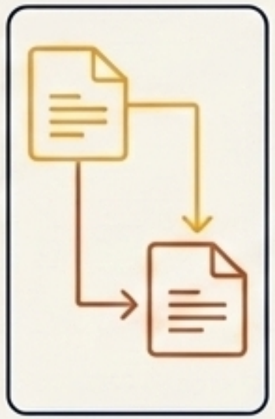
**O Gatilho:** Comando `/opsx: archive` detecta o Delta Spec (diferença entre a mudança atual e a spec oficial)

**Sincronizar Agora (Caminho Feliz)**



Copia os requisitos validados para a documentação permanente (`/specs/`) e move a mudança para o arquivo histórico.
A especificação torna-se oficial.

**Arquivar sem Sincronizar (Caminho Experimental)**

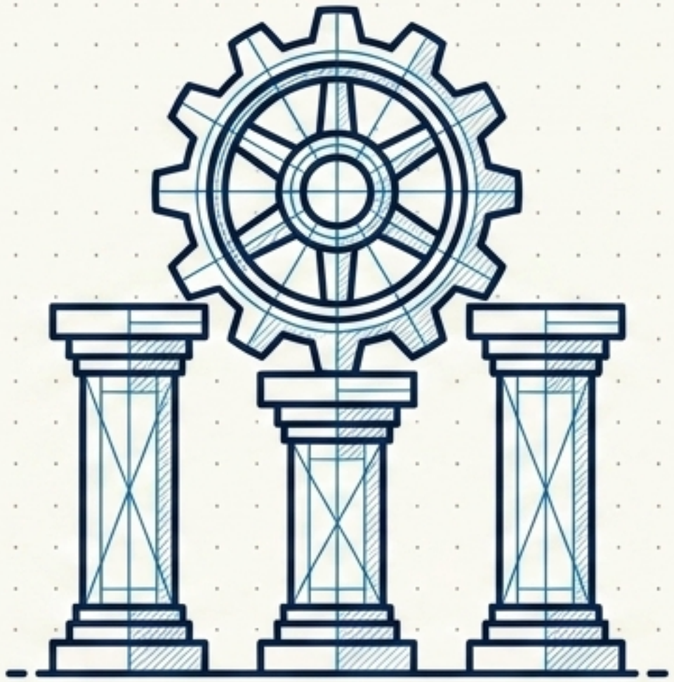


Move a mudança para o histórico (`archive`), mas deixa a documentação viva intacta.

Ideal para features descartadas ou experimentos.

Gemini Notebook

# O Estado Final de uma Feature



### Código Funcional e Previsível

A implementação física obedeceu regras estritas sem improvisações ou "alucinações" arquiteturais da IA.

### Documentação Viva (/specs/)

A fonte da verdade do sistema está totalmente sincronizada com o código recém-produzido, servindo de base para o próximo ciclo.

### Memória de Sistema (/archive/)

As decisões técnicas e o passo a passo da mudança estão documentados permanentemente no histórico técnico da aplicação.

Gemini Notebook

# OpenSpec Cheat Sheet

## Anatomia das Pastas

📁 **`.claude/`** : As Ferramentas (Como a IA opera)

📁 **`.openspec/`** : O Cérebro (Onde o projeto vive)

## O Ciclo Vital `/opsx`

**> `explorer`** : INVESTIGUE o terreno tecnológico.

**> `propose`** : PLANEJE a arquitetura (Gere Markdown).

**> `apply`** : EXECUTE o código (Siga o Tasks.md).

**> `archive`** : CONSOLIDE a documentação viva e arquive o histórico.

Gemini Notebook